

Final Exam (part 3) - Computational Physics I

Deadline: Thursday 5 June 2025 (by 23h59)

7/10

Credits: 10 points

Please keep the structure provided below and submit an organised notebook with clear answers to each item.

Name: Juan Daniel Vasconez

3. Wavelet analysis of seismic waves (10 points)

Seismic signals recorded at a single station typically comprise a superposition of different wave types, primarily **P-waves** (due to compression), **S-waves** (due to shear), and **surface waves** (the ones that cause ground shaking). These waves propagate through the Earth at different velocities and possess distinct characteristic frequencies, making their isolation crucial for accurate seismic event analysis and hazard assessment.

Dataset:

Please download the following data file: https://github.com/wbandabarragan/physics-teaching-data/blob/main/1D-data/seismic_signals.csv

This dataset contains a mock time series of seismic waveform amplitude from two different seismic stations. The time is recorded in seconds (s) and the wave amplitudes are in units of mm/s.

In []:  `import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import pywt`

Tasks:

The purpose of this problem is to develop a python pipeline (**using python functions**) that effectively isolates P-, S-, and surface waves from a given single-component seismic waveform using **wavelet transforms**, for which you need to carry out the following tasks:

(a) Read the data file and carry out a time-frequency analysis of the seismic signal of the first station using appropriate wavelet transforms. Compute the scalogram (i.e., the amplitude as a function of time and frequency) and the corresponding real frequencies.

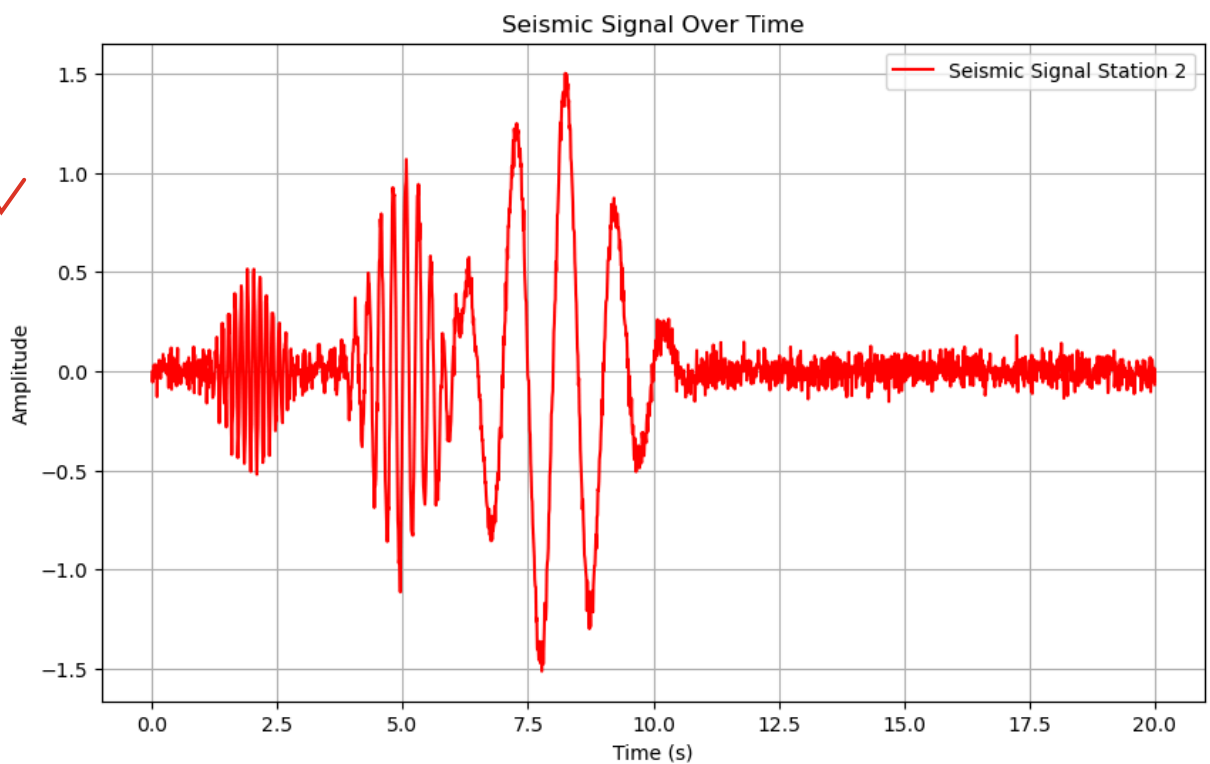
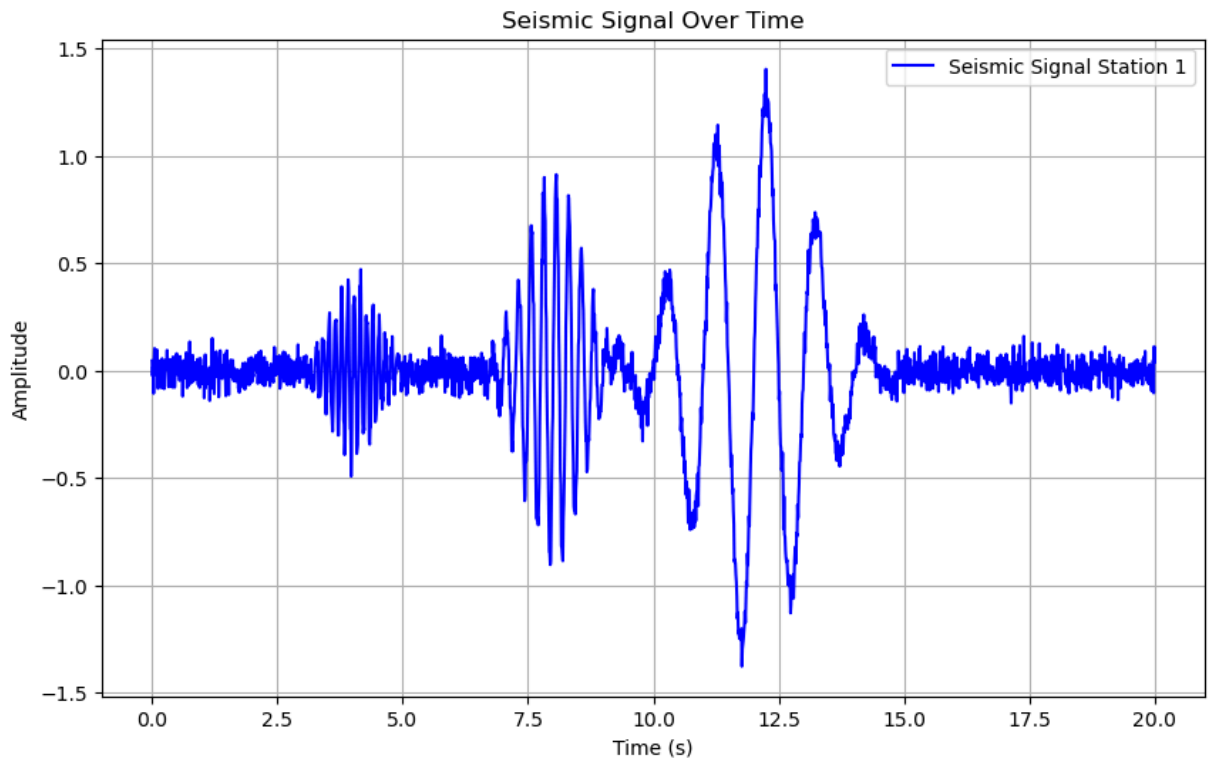
In [3]:  `df = pd.read_csv("seismic_signals.csv")`
`df.head(10)` **Comment out.**

Out [3]:

	Time (s)	Station1_Amplitude (mm/s)	Station2_Amplitude (mm/s)
0	0.000000	0.046240	-0.004670
1	0.010005	-0.023872	-0.055504
2	0.020010	0.019879	-0.017586
3	0.030015	-0.105718	-0.039203
4	0.040020	0.046148	0.030639
5	0.050025	0.104086	-0.012273
6	0.060030	0.090831	0.031852
7	0.070035	-0.015743	-0.002194
8	0.080040	0.068657	0.000254
9	0.090045	-0.018264	0.031209

In [4]:  `plt.figure(figsize=(10, 6))`
`plt.plot(df['Time (s)'], df['Station1_Amplitude (mm/s)'], label='Seismic Sig`
`plt.title('Seismic Signal Over Time')`
`plt.xlabel('Time (s)')`
`plt.ylabel('Amplitude')`
`plt.grid()`
`plt.legend()`
`plt.show()`

 `plt.figure(figsize=(10, 6))`
`plt.plot(df['Time (s)'], df['Station2_Amplitude (mm/s)'], label='Seismic Sig`
`plt.title('Seismic Signal Over Time')`
`plt.xlabel('Time (s)')`
`plt.ylabel('Amplitude')`
`plt.grid()`
`plt.legend()`
`plt.show()`



In []: `def wavelet_analysis(time_series, time, wavelet='cmor1.5-1.0', scales=None):`
 `.....`
 Perform continuous wavelet transform analysis on seismic data

Parameters:

- `time_series`: array-like, the seismic signal
- `time`: array-like, time values
- `wavelet`: str, wavelet to use for CWT
- `scales`: array-like, scales for CWT

✓

```

Returns:
coefficients: 2D array, wavelet coefficients
frequencies: array, corresponding frequencies
"""
# Calculate sampling frequency
dt = time[1] - time[0]
fs = 1 / dt

# Define scales if not provided
if scales is None:
    # Create scales from 1 to 250
    scales = np.arange(1, 250)

# Perform continuous wavelet transform
coefficients, frequencies = pywt.cwt(time_series, scales, wavelet, sample_rate)

return coefficients, frequencies, fs

# Extract Station 1 data
station1_signal = df['Station1_Amplitude (mm/s)'].values
time = df['Time (s)'].values

# Perform wavelet analysis
coefficients, frequencies, sampling_freq = wavelet_analysis(station1_signal,

# Calculate the scalogram (magnitude of coefficients)
scalogram = np.abs(coefficients)

print(f"Sampling frequency: {sampling_freq:.2f} Hz")
print(f"Time duration: {time[-1] - time[0]:.2f} seconds")
print(f"Frequency range: {frequencies.min():.3f} Hz to {frequencies.max():.3f} Hz")
print(f"Scalogram shape: {scalogram.shape}")

```

Sampling frequency: 99.95 Hz
 Time duration: 20.00 seconds
 Frequency range: 0.401 Hz to 99.950 Hz
 Scalogram shape: (249, 2000)

(b) Make two plots of the resulting 2D scalogram: one projection plot in 2D and another one as a 3D surface plot, ensuring both visualisations display the correct frequencies versus time. These plots are essential for visualising the frequency content of the different wave arrivals over time, which will inform the subsequent wave isolation process.

In [9]:

```

# Create 2D and 3D visualizations of the scalogram with better spacing
plt.figure(figsize=(15, 12))

# 2D projection plot
plt.subplot(2, 1, 1)
plt.imshow(scalogram, extent=[time[0], time[-1], frequencies[-1], frequencies[0]],
           cmap='turbo', aspect='auto', interpolation='bilinear')
plt.colorbar(label='Amplitude')
plt.xlabel('Time (s)')
plt.ylabel('Frequency (Hz)')

```

```

plt.title('2D Scalogram - Time-Frequency Analysis of Seismic Signal (Station 1)')
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

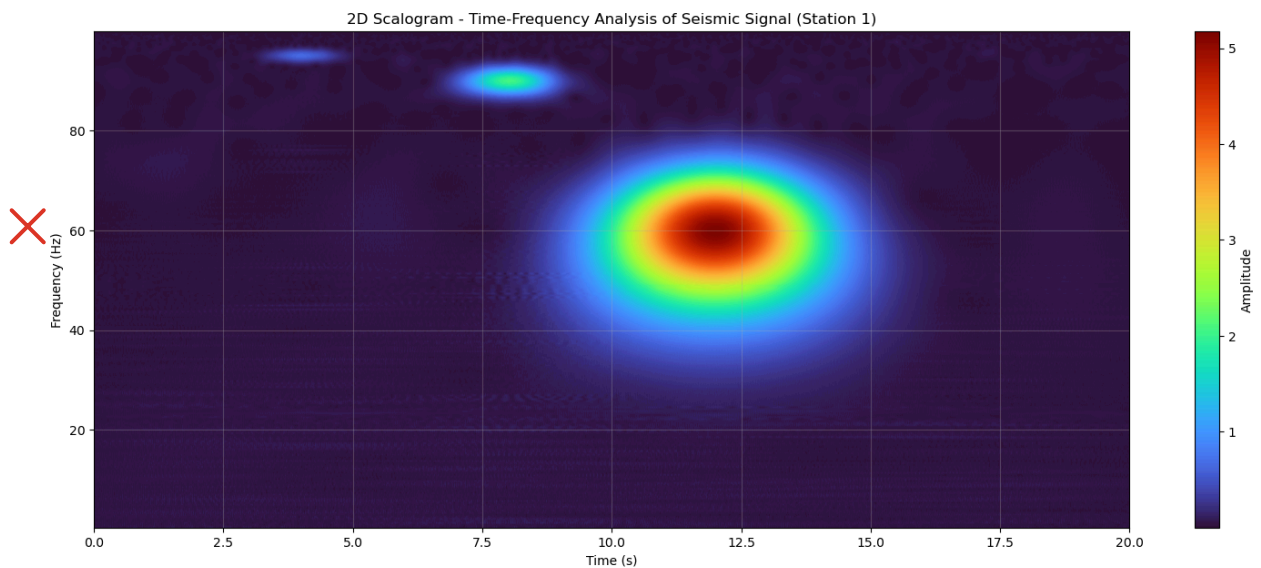
# 3D surface plot in a separate figure
fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111, projection='3d')

# Create mesh grids for 3D plotting
T, F = np.meshgrid(time, frequencies)
surface = ax.plot_surface(T, F, scalogram, cmap='turbo', alpha=0.8,
                          linewidth=0, antialiased=True)

ax.set_xlabel('Time (s)')
ax.set_ylabel('Frequency (Hz)')
ax.set_zlabel('Amplitude')
ax.set_title('3D Scalogram - Time-Frequency Analysis of Seismic Signal (Station 1)')
fig.colorbar(surface, ax=ax, shrink=0.5, aspect=10)

plt.tight_layout()
plt.show()

```



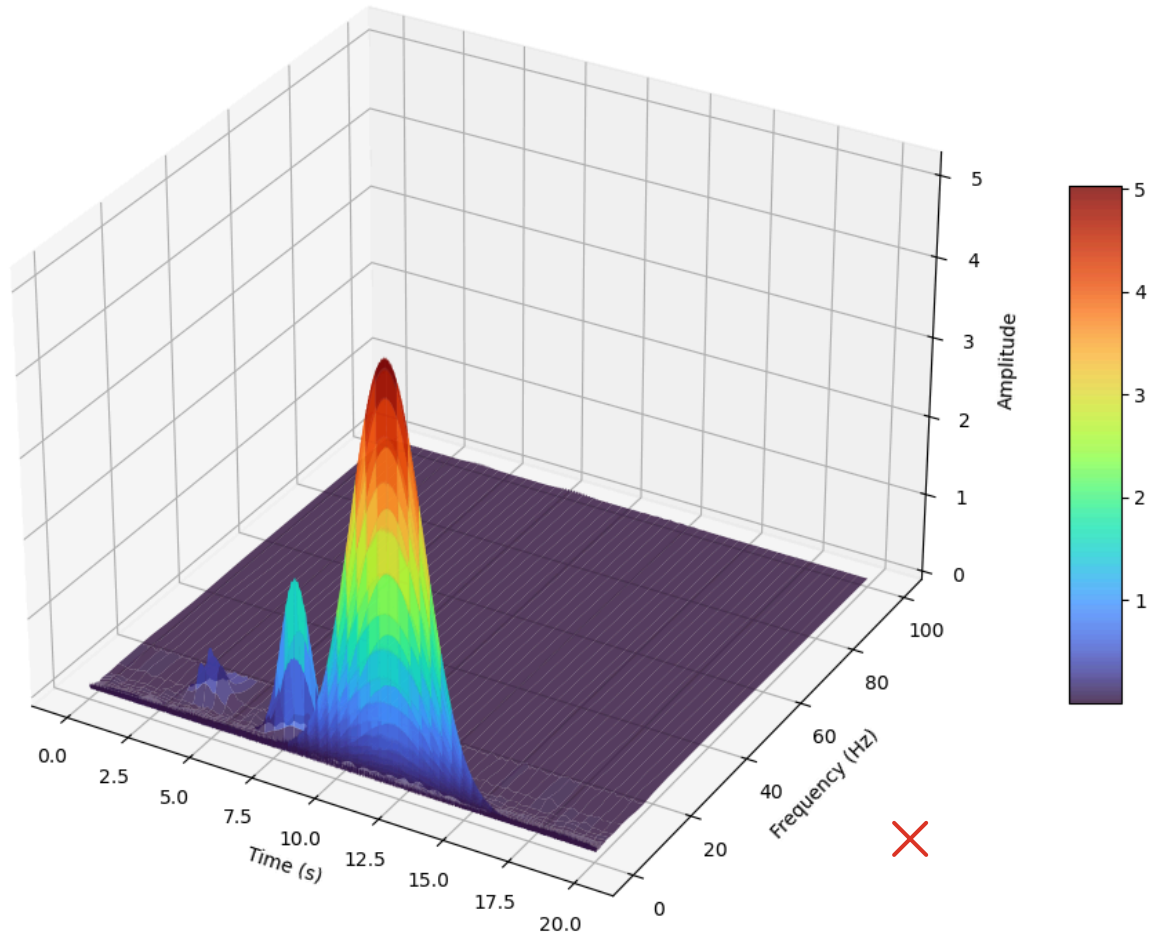
The frequencies seem high
and not correct..

-1.0

Increase font size.

-0.25

3D Scalogram - Time-Frequency Analysis of Seismic Signal (Station 1)



(c) Based on the insights gained from your scalogram plots, provide the approximate frequency ranges associated with the P-waves, S-waves, and surface waves observed in the seismic signal. Justify your identification with reference to the visual characteristics of the scalogram.

Explanation is Ok, but the frequencies are not correct.

The scalogram reveals a high-frequency energy burst at ~3–5 s (90–100 Hz), which is consistent with the expected characteristics of P-waves: early arrival and high-frequency content. ✓ A second energy packet appears at ~6.5–9.5 s (80–90 Hz), suggesting the arrival of S-waves, which typically follow P-waves and exhibit slightly lower frequencies. ✓ Finally, a broad, high-amplitude region between 8–16 s at ~30–80 Hz corresponds to surface waves, identifiable by their lower frequency content, larger amplitude, and longer duration. ✓

-0.25

```
In [28]: # Define time and frequency bounds for each wave type based on scalogram and
wave_windows = {
    'P-wave': {'time': (3, 5), 'freq': (90, 100)},
    'S-wave': {'time': (6.5, 9.5), 'freq': (80, 95)},
    'Surface-wave': {'time': (8, 16), 'freq': (30, 80)}
}
```

```

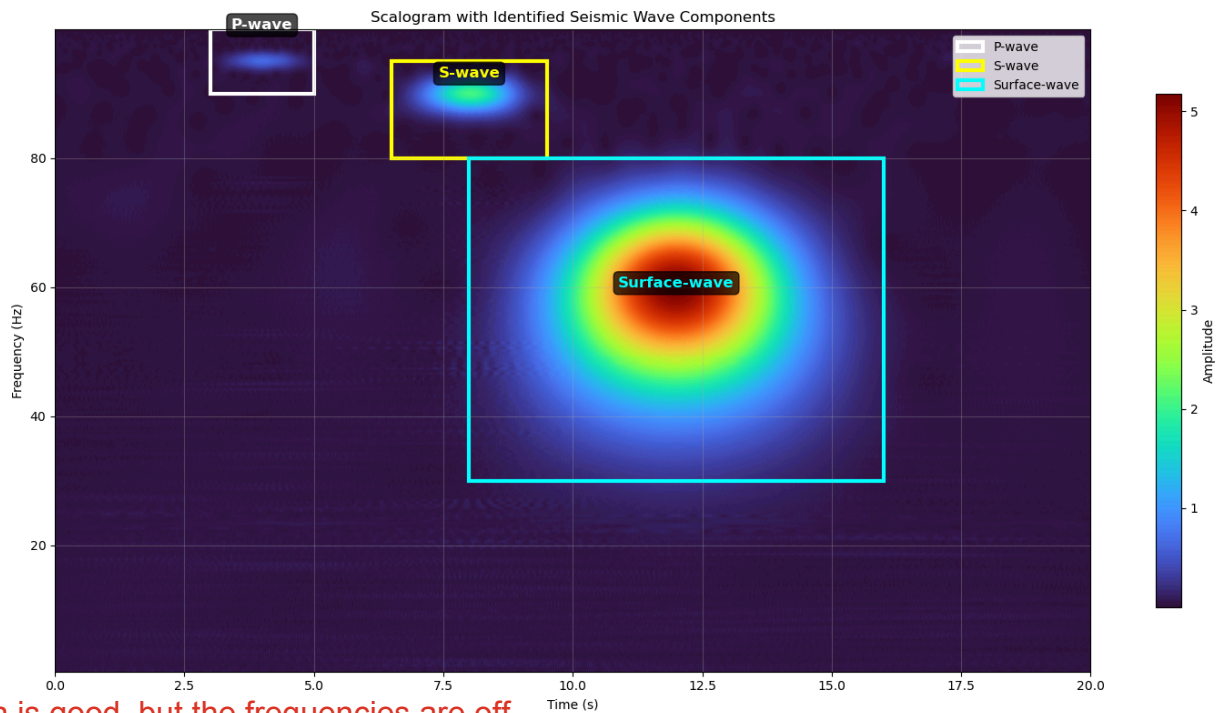
✓ # Visualization with annotated wave windows
plt.figure(figsize=(15, 8))
plt.imshow(scalogram, extent=[time[0], time[-1], frequencies[-1], frequencies[0]],
           cmap='turbo', aspect='auto', interpolation='bilinear')
plt.colorbar(label='Amplitude', shrink=0.8)
✓ plt.xlabel('Time (s)')
plt.ylabel('Frequency (Hz)')
plt.title('Scalogram with Identified Seismic Wave Components')

# Add rectangles and annotations for each wave window
colors = ['white', 'yellow', 'cyan']
for i, (wave_type, window) in enumerate(wave_windows.items()):
    # Draw rectangle
    ✓ rect = plt.Rectangle(
        (window['time'][0], window['freq'][0]),
        window['time'][1] - window['time'][0],
        window['freq'][1] - window['freq'][0],
        edgecolor=colors[i], linewidth=3, fill=False, label=wave_type
    )
    plt.gca().add_patch(rect)

    # Add text annotation
    ✓ plt.text(
        np.mean(window['time']),
        np.mean(window['freq']) + 5,
        wave_type,
        color=colors[i],
        fontsize=12,
        fontweight='bold',
        ha='center',
        bbox=dict(boxstyle="round,pad=0.3", facecolor='black', alpha=0.7)
    )

plt.legend(loc='upper right')
plt.grid(True, alpha=0.3)
✓ plt.tight_layout()
plt.show()

```



Visualisation is good, but the frequencies are off.

(d) Before performing a Discrete Wavelet Transform (DWT), determine the sampling frequency of the seismic signal. Then, for a chosen DWT decomposition level, calculate and state the theoretical frequency ranges corresponding to the approximation and all relevant detail coefficients.

Use FFT instead.

```
In [31]: def compute_dwt_frequency_ranges(time, max_level):
        """
        Compute sampling frequency and theoretical frequency ranges for DWT decomposition.

        Parameters:
        time: array-like, time values from the signal
        max_level: int, maximum decomposition level for DWT

        Returns:
        fs: float, sampling frequency
        nyquist: float, Nyquist frequency
        detail_ranges: list of tuples, frequency ranges for detail coefficients
        approx_range: float, upper frequency bound for approximation coefficient
        """
        # Calculate sampling parameters
        dt = time[1] - time[0] # time step
        fs = 1 / dt # sampling frequency
        nyquist = fs / 2 # Nyquist frequency max?

        # Calculate frequency ranges for detail coefficients
        ranges = []
        for j in range(1, max_level + 1):
            f_high = fs / (2 ** j) # upper frequency bound for detail Dj
            f_low = fs / (2 ** (j + 1)) # lower frequency bound for detail Dj
            ranges.append((f_low, f_high))

        # Frequency range for approximation coefficient
```

Check
sampling.



```

approx_freq = fs / (2 ** (max_level + 1))

# Display results
print(f"Sampling frequency: {fs:.2f} Hz")
print(f"Nyquist frequency: {nyquist:.2f} Hz\n")
print(f"Theoretical frequency ranges for DWT decomposition (Level {max_level + 1}):")
print("=" * 60)

# Print detail coefficient frequency ranges
for j, (f_low, f_high) in enumerate(ranges, start=1):
    print(f"D{j: <2} (Detail {j}): {f_low:6.2f} Hz - {f_high:6.2f} Hz")

# Print approximation coefficient frequency range
print(f"A{max_level} (Approximation): 0.00 Hz - {approx_freq:.2f} Hz")

return fs, nyquist, ranges, approx_freq

# Set decomposition level and compute frequency ranges
max_level = 7 # chosen decomposition level
fs, nyquist, detail_ranges, approx_range = compute_dwt_frequency_ranges(time

```

Sampling frequency: 99.95 Hz

Nyquist frequency: 49.98 Hz

Theoretical frequency ranges for DWT decomposition (Level 7):

```

=====
D1 (Detail 1): 24.99 Hz - 49.98 Hz
D2 (Detail 2): 12.49 Hz - 24.99 Hz
D3 (Detail 3): 6.25 Hz - 12.49 Hz
D4 (Detail 4): 3.12 Hz - 6.25 Hz
D5 (Detail 5): 1.56 Hz - 3.12 Hz
D6 (Detail 6): 0.78 Hz - 1.56 Hz
D7 (Detail 7): 0.39 Hz - 0.78 Hz
A7 (Approximation): 0.00 Hz - 0.39 Hz

```

These frequencies look better.

(e) Apply a suitable DWT to the seismic signal to decompose it into approximation and detail coefficients at multiple levels. The choice of wavelet family and decomposition level should be justified based on the characteristic frequencies of the seismic waves identified in the previous steps.

In [40]:

```

def apply_dwt_decomposition(signal, wavelet='db8', levels=7):
    """
    Apply Discrete Wavelet Transform to decompose seismic signal

    Parameters:
    signal: array-like, the seismic signal
    wavelet: str, wavelet family to use
    levels: int, number of decomposition levels

    Returns:
    coeffs: list, wavelet coefficients [cA_n, cD_n, cD_n-1, ..., cD_1]
    """
    coeffs = pywt.wavedec(signal, wavelet, level=levels)
    return coeffs

```



Ok.

```
# Apply DWT to Station 1 signal
# Using Daubechies 8 (db8) wavelet - good for seismic analysis due to:
# 1. Good time-frequency localization
# 2. Suitable for transient signals like seismic waves
# 3. Orthogonal and compactly supported
wavelet_family = 'db8'
coeffs_station1 = apply_dwt_decomposition(station1_signal, wavelet_family, n

print(f"DWT Decomposition Results (Wavelet: {wavelet_family}, Levels: {max_level})")
print("=" * 70)
print(f"Original signal length: {len(station1_signal)}")
print(f"Approximation coefficients (A{max_level}): {len(coeffs_station1[0])}")

for i in range(1, len(coeffs_station1)):
    detail_level = max_level - i + 1
    freq_range = detail_ranges[detail_level - 1]
    print(f"Detail coefficients D{detail_level}: {len(coeffs_station1[i])} samples, frequency range: {freq_range[0]:.2f} - {freq_range[1]:.2f} Hz")
```

✓

DWT Decomposition Results (Wavelet: db8, Levels: 7)

=====

Original signal length: 2000

Approximation coefficients (A7): 30 samples

Detail coefficients D7: 30 samples (0.39 - 0.78 Hz)

Detail coefficients D6: 46 samples (0.78 - 1.56 Hz)

Detail coefficients D5: 77 samples (1.56 - 3.12 Hz)

✓

Detail coefficients D4: 139 samples (3.12 - 6.25 Hz)

Detail coefficients D3: 263 samples (6.25 - 12.49 Hz)

Detail coefficients D2: 511 samples (12.49 - 24.99 Hz)

Detail coefficients D1: 1007 samples (24.99 - 49.98 Hz)

(f) Based on the understanding that different seismic wave types occupy distinct frequency bands (e.g., P-waves tend to have higher frequencies, S-waves mid-range, and surface waves lower frequencies), strategically manipulate the wavelet coefficients to reconstruct the isolated wave types. Specifically:

- **P-waves:** Reconstruct P-waves primarily from higher-frequency detail coefficients.
- **S-waves:** Reconstruct S-waves primarily from mid-frequency detail coefficients.
- **Surface Waves:** Reconstruct surface waves primarily from the lowest-frequency approximation coefficients.

```
In [33]: def isolate_seismic_waves(coeffs, wavelet='db8'):
        """
        Isolate different seismic wave types by manipulating DWT coefficients

        Parameters:
        coeffs: list, DWT coefficients [cA_n, cD_n, cD_n-1, ..., cD_1]
        wavelet: str, wavelet family used for decomposition

        Returns:
        p_wave_coeffs: list, coefficients for P-wave reconstruction
        s_wave_coeffs: list, coefficients for S-wave reconstruction
        surface_wave_coeffs: list, coefficients for surface wave reconstruction
        """
```

```

# Create coefficient arrays for each wave type (initialized with zeros)
p_wave_coeffs = [np.zeros_like(coeff) for coeff in coeffs]
s_wave_coeffs = [np.zeros_like(coeff) for coeff in coeffs]
surface_wave_coeffs = [np.zeros_like(coeff) for coeff in coeffs]

# P-waves: High frequencies (D1, D2) - frequencies ~25-100 Hz
# Based on frequency ranges: D1 (25-50 Hz), D2 (12.5-25 Hz)
p_wave_coeffs[-1] = coeffs[-1].copy() # D1: highest frequency details
p_wave_coeffs[-2] = coeffs[-2].copy() # D2: second highest frequency de

# S-waves: Mid frequencies (D2, D3, D4) - frequencies ~3-25 Hz
# Based on frequency ranges: D2 (12.5-25 Hz), D3 (6.25-12.5 Hz), D4 (3.1
s_wave_coeffs[-2] = coeffs[-2].copy() # D2
s_wave_coeffs[-3] = coeffs[-3].copy() # D3
s_wave_coeffs[-4] = coeffs[-4].copy() # D4

# Surface waves: Low frequencies (Approximation + D5, D6, D7)
# Based on frequency ranges: A7 (0-0.39 Hz), D7 (0.39-0.78 Hz), D6 (0.78
surface_wave_coeffs[0] = coeffs[0].copy() # A7: approximation (lowest
surface_wave_coeffs[1] = coeffs[1].copy() # D7
surface_wave_coeffs[2] = coeffs[2].copy() # D6
surface_wave_coeffs[3] = coeffs[3].copy() # D5

return p_wave_coeffs, s_wave_coeffs, surface_wave_coeffs

# Apply wave isolation to Station 1 coefficients
p_wave_coeffs, s_wave_coeffs, surface_wave_coeffs = isolate_seismic_waves(cc

# Display the frequency bands used for each wave type
print("Seismic Wave Isolation Strategy:")
print("=" * 50)
print("P-waves (High frequencies):")
print(f" - D1: {detail_ranges[0][0]::2f} - {detail_ranges[0][1]::2f} Hz")
print(f" - D2: {detail_ranges[1][0]::2f} - {detail_ranges[1][1]::2f} Hz")

print("\nS-waves (Mid frequencies):")
print(f" - D2: {detail_ranges[1][0]::2f} - {detail_ranges[1][1]::2f} Hz")
print(f" - D3: {detail_ranges[2][0]::2f} - {detail_ranges[2][1]::2f} Hz")
print(f" - D4: {detail_ranges[3][0]::2f} - {detail_ranges[3][1]::2f} Hz")

print("\nSurface waves (Low frequencies):")
print(f" - A7: 0.00 - {approx_range::2f} Hz")
print(f" - D7: {detail_ranges[6][0]::2f} - {detail_ranges[6][1]::2f} Hz")
print(f" - D6: {detail_ranges[5][0]::2f} - {detail_ranges[5][1]::2f} Hz")
print(f" - D5: {detail_ranges[4][0]::2f} - {detail_ranges[4][1]::2f} Hz")

# Verify coefficient manipulation
print(f"\nCoefficient manipulation verification:")
print(f"Original coefficients: {len(coeffs_station1)} levels")
print(f"P-wave non-zero levels: {sum(1 for c in p_wave_coeffs if np.any(c !=
print(f"S-wave non-zero levels: {sum(1 for c in s_wave_coeffs if np.any(c !=
print(f"Surface-wave non-zero levels: {sum(1 for c in surface_wave_coeffs if

```

Seismic Wave Isolation Strategy:

=====

P-waves (High frequencies):

-0.25

- D1: 24.99 – 49.98 Hz
- D2: 12.49 – 24.99 Hz



S-waves (Mid frequencies):

- D2: 12.49 – 24.99 Hz
- D3: 6.25 – 12.49 Hz
- D4: 3.12 – 6.25 Hz

P-waves.

Surface waves (Low frequencies):

- A7: 0.00 – 0.39 Hz
- D7: 0.39 – 0.78 Hz
- D6: 0.78 – 1.56 Hz
- D5: 1.56 – 3.12 Hz



These frequencies seem more correct.

Coefficient manipulation verification:

Original coefficients: 8 levels

P-wave non-zero levels: 2

S-wave non-zero levels: 3

Surface-wave non-zero levels: 4

(g) Reconstruct the isolated P-, S-, and surface wave signals using the modified wavelet coefficients.

```
In [35]: def reconstruct_wave_signals(p_coeffs, s_coeffs, surface_coeffs, wavelet='db
        """
        Reconstruct wave signals from modified DWT coefficients

        Parameters:
        p_coeffs, s_coeffs, surface_coeffs: list, modified DWT coefficients
        wavelet: str, wavelet family used for decomposition

        Returns:
        p_wave_signal: array, reconstructed P-wave signal
        s_wave_signal: array, reconstructed S-wave signal
        surface_wave_signal: array, reconstructed surface wave signal
        """
        # Reconstruct signals using inverse DWT
        p_wave_signal = pywt.waverec(p_coeffs, wavelet)
        s_wave_signal = pywt.waverec(s_coeffs, wavelet)
        surface_wave_signal = pywt.waverec(surface_coeffs, wavelet)

        return p_wave_signal, s_wave_signal, surface_wave_signal

    # Reconstruct the isolated wave signals
    p_wave_reconstructed, s_wave_reconstructed, surface_wave_reconstructed = rec
        p_wave_coeffs, s_wave_coeffs, surface_wave_coeffs, wavelet_family
    )

    # Verify reconstruction lengths match original signal
    print("Signal Reconstruction Results:")
    print("=" * 40)
    print(f"Original signal length: {len(station1_signal)}")
```

```

print(f"P-wave reconstruction length: {len(p_wave_reconstructed)}")
print(f"S-wave reconstruction length: {len(s_wave_reconstructed)}")
print(f"Surface-wave reconstruction length: {len(surface_wave_reconstructed)}")

# Calculate reconstruction statistics
print(f"\nSignal Statistics:")
print(f"P-wave amplitude range: {p_wave_reconstructed.min():.4f} to {p_wave_")
print(f"S-wave amplitude range: {s_wave_reconstructed.min():.4f} to {s_wave_")
print(f"Surface-wave amplitude range: {surface_wave_reconstructed.min():.4f}")

# Verify energy conservation (sum of reconstructed signals should approximate total)
total_reconstruction = p_wave_reconstructed + s_wave_reconstructed + surface_")
reconstruction_error = np.mean(np.abs(station1_signal - total_reconstruction))
print(f"\nReconstruction quality:")
print(f"Mean absolute error: {reconstruction_error:.6f}")
print(f"Relative error: {100 * reconstruction_error / np.std(station1_signal)}")

```

Signal Reconstruction Results:

=====

Original signal length: 2000
P-wave reconstruction length: 2000
S-wave reconstruction length: 2000
Surface-wave reconstruction length: 2000

Signal Statistics:

P-wave amplitude range: -0.1439 to 0.1438
S-wave amplitude range: -1.0231 to 0.9406
Surface-wave amplitude range: -1.2728 to 1.2792

Reconstruction quality:

Mean absolute error: 0.021152
Relative error: 6.40%

(h) Make a multi-panel figure of the first seismic waveform alongside the separated P-, S-, and surface wave components to demonstrate the effectiveness of the pipeline.

```

In [38]: plt.figure(figsize=(16, 12))

# Original seismic signal
plt.subplot(4, 1, 1)
plt.plot(time, station1_signal, 'k-', linewidth=1.5, label='Original Seismic')
plt.title('Station 1: Original Seismic Signal and Separated Wave Components')
plt.ylabel('Amplitude\n(mm/s)', fontsize=12)
plt.grid(True, alpha=0.3)
plt.legend(loc='upper right')
plt.xlim(0, 20)

# P-wave reconstruction
plt.subplot(4, 1, 2)
plt.plot(time, p_wave_reconstructed, 'r-', linewidth=1.5, label='P-waves (H')
plt.ylabel('Amplitude\n(mm/s)', fontsize=12)
plt.grid(True, alpha=0.3)
plt.legend(loc='upper right')
plt.xlim(0, 20)

# S-wave reconstruction

```

```

plt.subplot(4, 1, 3)
plt.plot(time, s_wave_reconstructed, 'b-', linewidth=1.5, label='S-waves (Mi
plt.ylabel('Amplitude\n(mm/s)', fontsize=12)
plt.grid(True, alpha=0.3)
plt.legend(loc='upper right')
plt.xlim(0, 20)

# Surface wave reconstruction
plt.subplot(4, 1, 4)
plt.plot(time, surface_wave_reconstructed, 'g-', linewidth=1.5, label='Surfa
plt.xlabel('Time (s)', fontsize=12)
plt.ylabel('Amplitude\n(mm/s)', fontsize=12)
plt.grid(True, alpha=0.3)
plt.legend(loc='upper right')
plt.xlim(0, 20)

plt.tight_layout()
plt.show()

# Additional validation plot showing reconstruction quality
plt.figure(figsize=(16, 10))

# Top subplot: Original vs Total reconstruction
plt.subplot(3, 1, 1)
plt.plot(time, station1_signal, 'k-', linewidth=2, label='Original Signal',
plt.plot(time, total_reconstruction, 'r--', linewidth=2, label='Sum of Recor
plt.title('Reconstruction Quality Assessment', fontsize=14, fontweight='bold
plt.ylabel('Amplitude (mm/s)', fontsize=12)
plt.legend()
plt.grid(True, alpha=0.3)
plt.xlim(0, 20)

# Middle subplot: Reconstruction error
plt.subplot(3, 1, 2)
reconstruction_diff = station1_signal - total_reconstruction
plt.plot(time, reconstruction_diff, 'purple', linewidth=1.5, label=f'Reconst
plt.ylabel('Error\n(mm/s)', fontsize=12)
plt.legend()
plt.grid(True, alpha=0.3)
plt.xlim(0, 20)

# Bottom subplot: All wave components overlaid
plt.subplot(3, 1, 3)
plt.plot(time, p_wave_reconstructed, 'r-', linewidth=1.5, label='P-waves', a
plt.plot(time, s_wave_reconstructed, 'b-', linewidth=1.5, label='S-waves', a
plt.plot(time, surface_wave_reconstructed, 'g-', linewidth=1.5, label='Surfa
plt.plot(time, station1_signal, 'k-', linewidth=1, label='Original', alpha=0
plt.xlabel('Time (s)', fontsize=12)
plt.ylabel('Amplitude (mm/s)', fontsize=12)
plt.title('Wave Component Comparison', fontsize=12)
plt.legend()
plt.grid(True, alpha=0.3)
plt.xlim(0, 20)

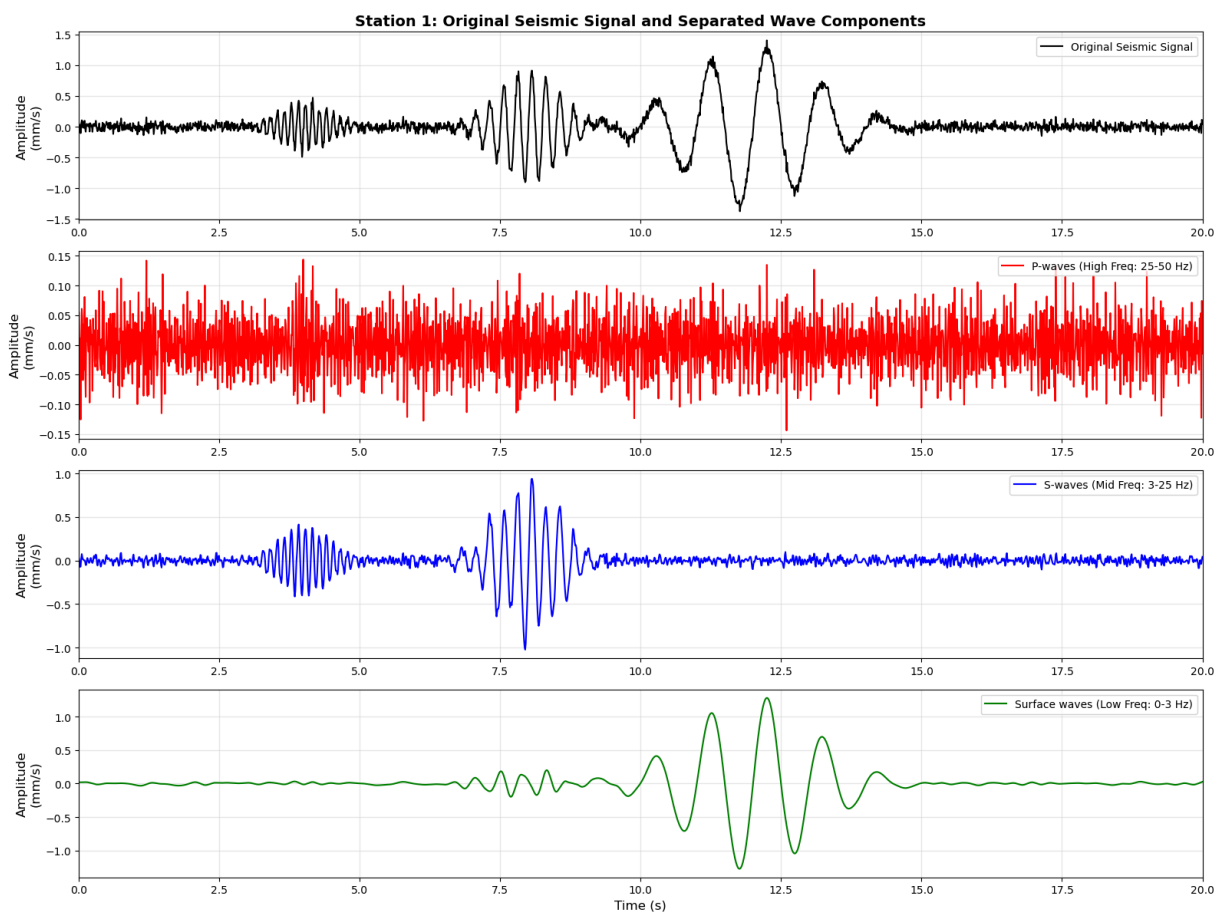
plt.tight_layout()
plt.show()

```

```
# Print summary of wave isolation results
print("Wave Isolation Summary for Station 1:")
print("=" * 50)
print(f"P-wave energy (RMS): {np.sqrt(np.mean(p_wave_reconstructed**2)):.4f}")
print(f"S-wave energy (RMS): {np.sqrt(np.mean(s_wave_reconstructed**2)):.4f}")
print(f"Surface-wave energy (RMS): {np.sqrt(np.mean(surface_wave_reconstructed**2)):.4f}")
print(f"Total reconstruction error: {100 * reconstruction_error / np.std(station1_signal):.2f}%")

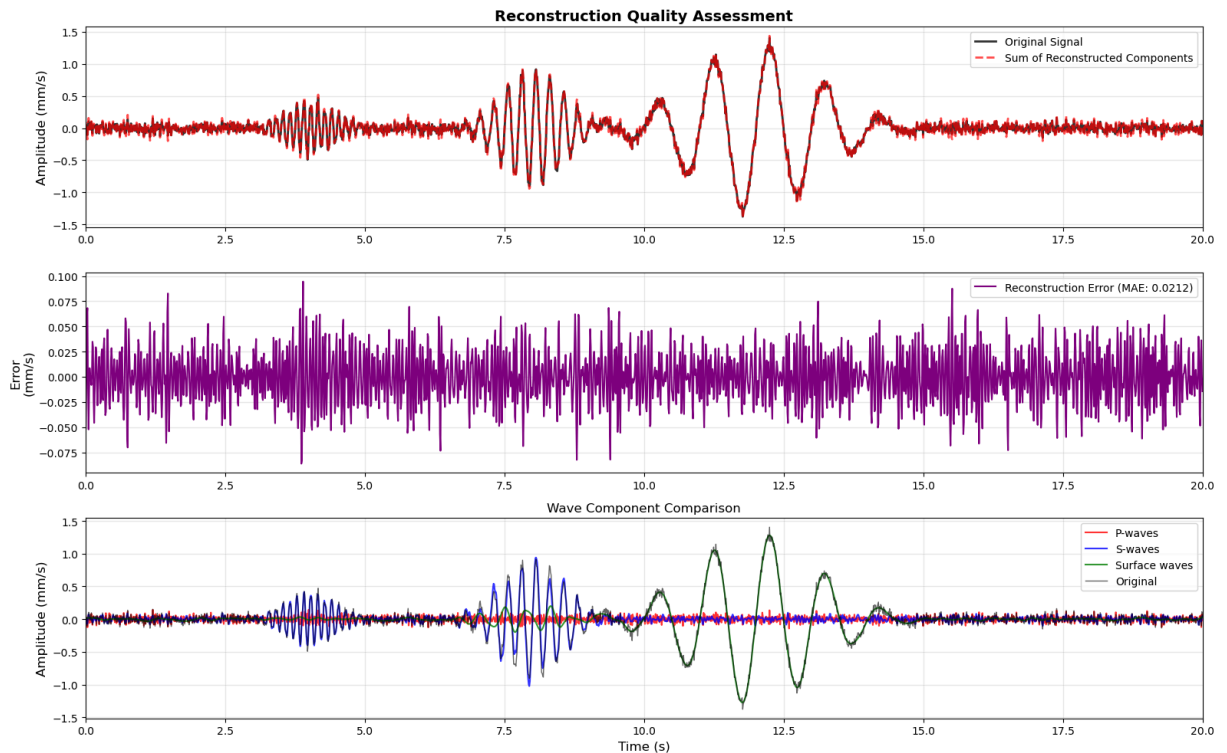
total_energy = np.sum(station1_signal**2)
p_energy = np.sum(p_wave_reconstructed**2)
s_energy = np.sum(s_wave_reconstructed**2)
surface_energy = np.sum(surface_wave_reconstructed**2)

print(f"P-wave relative energy: {100 * p_energy / total_energy:.2f}%")
print(f"S-wave relative energy: {100 * s_energy / total_energy:.2f}%")
print(f"Surface-wave relative energy: {100 * surface_energy / total_energy:.2f}%")
```



?

✗



Wave Isolation Summary for Station 1:

```
=====
P-wave energy (RMS): 0.0439
S-wave energy (RMS): 0.1508
Surface-wave energy (RMS): 0.2918
Total reconstruction error: 6.40%
P-wave relative energy: 1.76%
S-wave relative energy: 20.83%
Surface-wave relative energy: 78.03%
```

(i) Validate the code by applying the pipeline to the data of the second station. Make a multi-panel figure of the second seismic waveform alongside the separated P-, S-, and surface wave components to demonstrate the effectiveness of the pipeline.

```
In [41]: # Extract Station 2 data
station2_signal = df['Station2_Amplitude (mm/s)'].values

# Apply DWT decomposition to Station 2
coeffs_station2 = apply_dwt_decomposition(station2_signal, wavelet_family, n

print(f"Station 2 DWT Decomposition Results:")
print(f"Original signal length: {len(station2_signal)}")
print(f"Approximation coefficients (A{max_level}): {len(coeffs_station2[0])}")

# Apply wave isolation to Station 2 coefficients
p_wave_coeffs_s2, s_wave_coeffs_s2, surface_wave_coeffs_s2 = isolate_seismic

# Reconstruct the isolated wave signals for Station 2
p_wave_reconstructed_s2, s_wave_reconstructed_s2, surface_wave_reconstructed_s2 = reconstruct_signals(
    p_wave_coeffs_s2, s_wave_coeffs_s2, surface_wave_coeffs_s2, wavelet_fami
)

# Calculate reconstruction statistics for Station 2
```

✓

✓

✓


```

total_reconstruction_s2 = p_wave_reconstructed_s2 + s_wave_reconstructed_s2
reconstruction_error_s2 = np.mean(np.abs(station2_signal - total_reconstruct

print(f"Station 2 Reconstruction quality:")
print(f"Mean absolute error: {reconstruction_error_s2:.6f}")
print(f"Relative error: {100 * reconstruction_error_s2 / np.std(station2_sig

# Create multi-panel figure for Station 2
plt.figure(figsize=(16, 12))

# Original seismic signal for Station 2
plt.subplot(4, 1, 1)
plt.plot(time, station2_signal, 'k-', linewidth=1.5, label='Original Seismic')
plt.title('Station 2: Original Seismic Signal and Separated Wave Components')
plt.ylabel('Amplitude\n(mm/s)', fontsize=12)
plt.grid(True, alpha=0.3)
plt.legend(loc='upper right')
plt.xlim(0, 20)

# P-wave reconstruction for Station 2
plt.subplot(4, 1, 2)
plt.plot(time, p_wave_reconstructed_s2, 'r-', linewidth=1.5, label='P-waves')
plt.ylabel('Amplitude\n(mm/s)', fontsize=12)
plt.grid(True, alpha=0.3)
plt.legend(loc='upper right')
plt.xlim(0, 20)

# S-wave reconstruction for Station 2
plt.subplot(4, 1, 3)
plt.plot(time, s_wave_reconstructed_s2, 'b-', linewidth=1.5, label='S-waves')
plt.ylabel('Amplitude\n(mm/s)', fontsize=12)
plt.grid(True, alpha=0.3)
plt.legend(loc='upper right')
plt.xlim(0, 20)

# Surface wave reconstruction for Station 2
plt.subplot(4, 1, 4)
plt.plot(time, surface_wave_reconstructed_s2, 'g-', linewidth=1.5, label='Su')
plt.xlabel('Time (s)', fontsize=12)
plt.ylabel('Amplitude\n(mm/s)', fontsize=12)
plt.grid(True, alpha=0.3)
plt.legend(loc='upper right')
plt.xlim(0, 20)

plt.tight_layout()
plt.show()

# Additional validation plot for Station 2
plt.figure(figsize=(16, 10))

# Top subplot: Original vs Total reconstruction
plt.subplot(3, 1, 1)
plt.plot(time, station2_signal, 'k-', linewidth=2, label='Original Signal',
plt.plot(time, total_reconstruction_s2, 'r--', linewidth=2, label='Sum of Re
plt.title('Station 2: Reconstruction Quality Assessment', fontsize=14, fontw
plt.ylabel('Amplitude (mm/s)', fontsize=12)

```

```

plt.legend()
plt.grid(True, alpha=0.3)
plt.xlim(0, 20)

# Middle subplot: Reconstruction error
plt.subplot(3, 1, 2)
reconstruction_diff_s2 = station2_signal - total_reconstruction_s2
plt.plot(time, reconstruction_diff_s2, 'purple', linewidth=1.5, label=f'Reconstruction Error')
plt.ylabel('Error\n(mm/s)', fontsize=12)
plt.legend()
plt.grid(True, alpha=0.3)
plt.xlim(0, 20)

# Bottom subplot: All wave components overlaid
plt.subplot(3, 1, 3)
plt.plot(time, p_wave_reconstructed_s2, 'r-', linewidth=1.5, label='P-waves')
plt.plot(time, s_wave_reconstructed_s2, 'b-', linewidth=1.5, label='S-waves')
plt.plot(time, surface_wave_reconstructed_s2, 'g-', linewidth=1.5, label='Surface waves')
plt.plot(time, station2_signal, 'k-', linewidth=1, label='Original', alpha=0.5)
plt.xlabel('Time (s)', fontsize=12)
plt.ylabel('Amplitude (mm/s)', fontsize=12)
plt.title('Wave Component Comparison', fontsize=12)
plt.legend()
plt.grid(True, alpha=0.3)
plt.xlim(0, 20)

plt.tight_layout()
plt.show()

# Calculate energy statistics for Station 2
total_energy_s2 = np.sum(station2_signal**2)
p_energy_s2 = np.sum(p_wave_reconstructed_s2**2)
s_energy_s2 = np.sum(s_wave_reconstructed_s2**2)
surface_energy_s2 = np.sum(surface_wave_reconstructed_s2**2)

# Print summary of wave isolation results for Station 2
print("\nWave Isolation Summary for Station 2:")
print("=" * 50)
print(f"P-wave energy (RMS): {np.sqrt(np.mean(p_wave_reconstructed_s2**2)):.2f} mm/s")
print(f"S-wave energy (RMS): {np.sqrt(np.mean(s_wave_reconstructed_s2**2)):.2f} mm/s")
print(f"Surface-wave energy (RMS): {np.sqrt(np.mean(surface_wave_reconstructed_s2**2)):.2f} mm/s")
print(f"Total reconstruction error: {100 * reconstruction_error_s2 / np.std(station2_signal):.2f}%")

print(f"P-wave relative energy: {100 * p_energy_s2 / total_energy_s2:.2f}%")
print(f"S-wave relative energy: {100 * s_energy_s2 / total_energy_s2:.2f}%")
print(f"Surface-wave relative energy: {100 * surface_energy_s2 / total_energy_s2:.2f}%")

# Comparison between stations
print(f"\nComparison between Station 1 and Station 2:")
print("=" * 50)
print(f"Station 1 reconstruction error: {100 * reconstruction_error / np.std(station1_signal):.2f}%")
print(f"Station 2 reconstruction error: {100 * reconstruction_error_s2 / np.std(station2_signal):.2f}%")

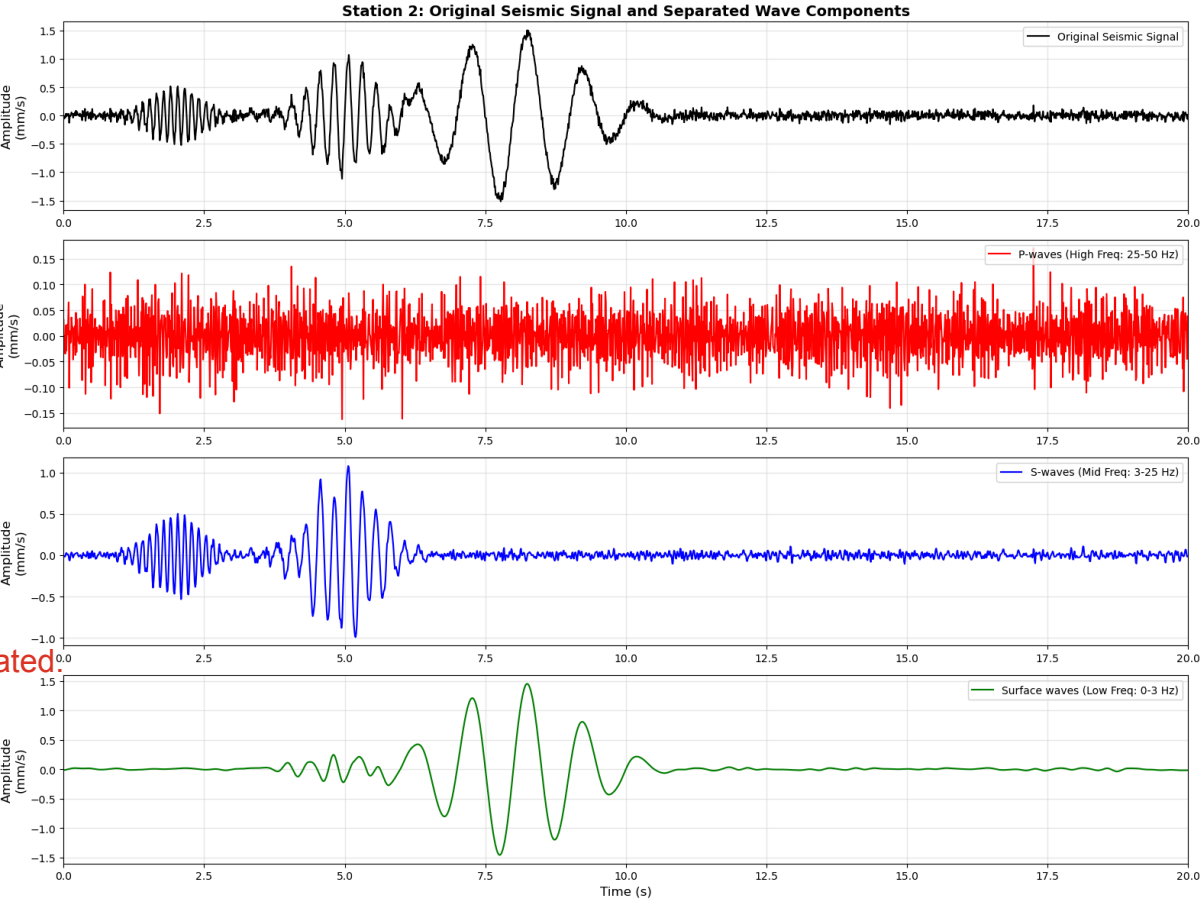
```

Station 2 DWT Decomposition Results:
Original signal length: 2000
Approximation coefficients (A7): 30 samples
Station 2 Reconstruction quality:
Mean absolute error: 0.019736
Relative error: 5.23%

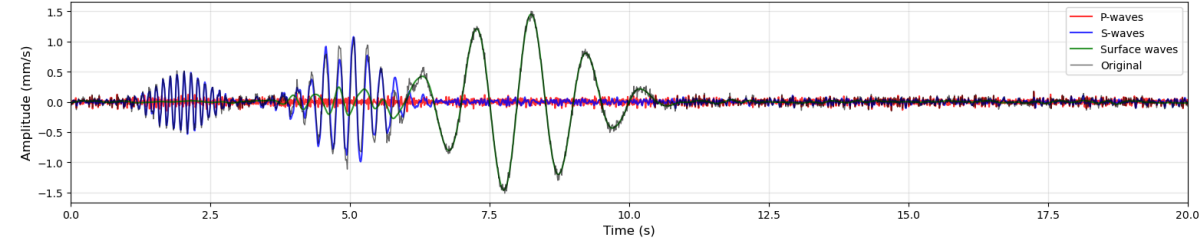
×

-0.25

Not fully separated



?



Wave Isolation Summary for Station 2:

=====

P-wave energy (RMS): 0.0450

S-wave energy (RMS): 0.1708

Surface-wave energy (RMS): 0.3344

Total reconstruction error: 5.23%

P-wave relative energy: 1.42%

S-wave relative energy: 20.48%

Surface-wave relative energy: 78.54%

Comparison between Station 1 and Station 2:

=====

Station 1 reconstruction error: 6.40%

Station 2 reconstruction error: 5.23%

(j) Does your code successfully isolate the different wave components of the seismic signals?

- YES **Expand analysis.**

-0.75

The frequencies are not correct.

DWT is decomposing using the correct frequencies, but the CWT is not correct.